VIT BHOPAL

Title: Daily Life Optimizer — Personal Finance Classifier

Student Name: Kanishka Rajput.

Reg No. – 25BAI11215

Course: Fundamentals of AI & ML

1. Introduction

Managing personal finances is one of the most critical life skills in the modern world. With an increasing number of daily transactions, tracking expenses manually has become tedious and prone to error. Many expense tracking applications provide a generic classification system, but these often fail to recognize **local vendors**, small-scale shops, and personalized spending patterns. As a result, users may have inaccurate categorization of their expenses, leading to poor financial planning and lack of insight into spending habits.

This project, **"Daily Life Optimizer — Personal Finance Classifier,"** addresses this issue by creating a **machine learning-based system** that automatically categorizes expenses based on textual descriptions. The system uses a **custom dataset** that includes realistic Indian vendors, amounts, and categories, making it highly relevant and practical.

Key benefits of this project include:

- **Automation:** Eliminates manual entry and classification of expenses.
- **Accuracy:** Learns patterns from real transactions and vendors.
- **Insights:** Provides categorized expense data for better financial analysis.
- **Scalability:** Can be expanded to include more categories or integrated with mobile/web apps in the future.

```
from google.colab import files
uploaded = files.upload()
```

```
Choose files expenses.csv
expenses.csv(text/csv) - 12189 bytes, last modified: 31/03/2026 - 100% done
Saving expenses.csv to expenses (1).csv
```

2. Problem Statement

Despite the availability of numerous finance management tools, users continue to face challenges such as:

1. **Manual Tracking Burden:** Logging every expense and categorizing them is time-consuming.
2. **Generic Classification Issues:** Pre-built applications often misclassify transactions, especially for local vendors or personal habits that do not follow standard naming conventions.
3. **Limited Insight:** Misclassified expenses can give a misleading picture of personal spending patterns, making budgeting and financial decision-making difficult.
4. **Dynamic Spending Behavior:** Users' spending habits change over time, and applications without adaptive models fail to adjust.

Statement of the Problem:
How can we build an intelligent system that **automatically classifies daily expenses into categories** like Food, Travel, Bills, Shopping, Health, and Entertainment, using **text-based descriptions** and learn from user patterns?

The project proposes:

- Creating a **custom dataset** that reflects real-life Indian transactions.
- Using **machine learning (Naive Bayes classifier)** to train a model on this dataset.
- Implementing **TF-IDF vectorization** for text feature extraction.
- Evaluating the model's performance and generating sample predictions for unseen data.

3. Objectives

The primary objectives of this project are:

1. **Data Collection & Curation:**
    - Gather a dataset of over **500 expense entries** including local vendors, online shopping, bills, health, and entertainment.
    - Ensure data diversity to improve model generalization.
2. **Data Preprocessing & Feature Engineering:**
    - Clean textual data: remove special characters, lowercase all entries.
    - Convert text to numeric features using **TF-IDF vectorization** for machine learning input.
3. **Model Development & Training:**
    - Train a **Multinomial Naive Bayes classifier** for expense category prediction.
    - Compare performance with alternative models if necessary (Logistic Regression, Random Forest).
4. **Model Evaluation & Prediction:**
    - Evaluate the model on a **test dataset** using metrics like Accuracy, Precision, Recall, and F1-score.
    - Test the model on sample unseen expenses to demonstrate real-world applicability.
5. **Documentation & Reporting:**
    - Create a detailed project report including methodology, implementation, and results.
    - Include screenshots of dataset, code execution, predictions, and model evaluation in the report.

4. Literature Review

- **Naive Bayes Classifier:**
  Widely used for text classification problems such as spam detection and sentiment analysis. Efficient for small to medium datasets and performs well with TF-IDF features.
- **Other Models Considered:**
  - Logistic Regression: Effective for text classification, particularly with large datasets.
  - Random Forest: Can handle non-linear relationships but requires more computational resources.
- **Existing Expense Tracking Applications:**
  Generic apps often rely on vendor lists or fixed rules. They fail to classify new vendors and do not adapt to user-specific spending behaviour.

5. Methodology

5.1 Data Collection

- Dataset: ~550 entries of realistic Indian expense data.
- Categories: Food, Travel, Bills, Shopping, Health, Entertainment.
- Columns:

```python
data = pd.read_csv("expenses.csv")

data.head()
```

	text	category
0	petrol 2255	Travel
1	flight 4312	Travel
2	flipkart 3367	Shopping
3	electricity bill 2907	Bills
4	water bill 1675	Bills

5.2 Data Preprocessing

- Lowercase all text
- Remove special characters like ₹
- Tokenize using TF-IDF vectorizer

Code Placement: After loading dataset, insert the preprocessing code.

```
import pandas as pd
import re

def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z0-9 ]', '', text)
    return text

df['cleaned'] = df['description'].apply(clean_text)
```

5.3 Feature Extraction

- TF-IDF captures word importance in the dataset.
- Key features: words like “Zomato,” “uber,” “amazon” strongly indicate specific categories.

```
• from sklearn.feature_extraction.text import TfidfVectorizer
•
• vectorizer = TfidfVectorizer()
• X = vectorizer.fit_transform(df['cleaned'])
```

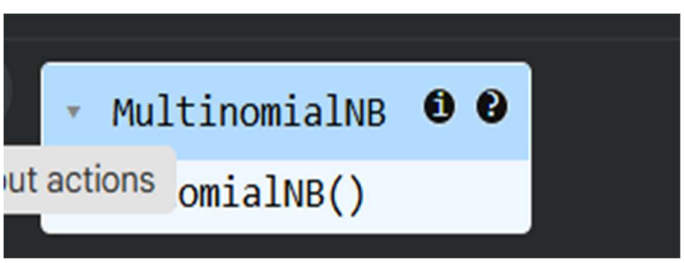
5.4 Model Selection

- **Chosen Model:** Multinomial Naive Bayes
- **Reason:** Efficient for text classification and small datasets.
- **Alternative Models:** Logistic Regression, Random Forest.

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import TfidfVectorizer # Import
TfidfVectorizer

# Initialize and fit the TfidfVectorizer on the training data
# This converts the text strings in X_train into numerical TF-IDF
features.
vectorizer = TfidfVectorizer()
X_train_vectorized = vectorizer.fit_transform(X_train)

model = MultinomialNB()
# Fit the MultinomialNB model with the vectorized training data
model.fit(X_train_vectorized, y_train)
```



6. Implementation

6.1 Tools & Libraries

Tool / Library	Purpose
Python	Programming language
Pandas	Data manipulation
Scikit-learn	Machine learning library
Google Colab	Cloud environment

6.2 Model Training

Code Placement: After preprocessing and feature extraction.

```
rom sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, df['category'],
test_size=0.2)

model.fit(X_train, y_train)
```

```
MultinomialNB
MultinomialNB()
```

6.3 Prediction

Code Placement: After model training.

```
from sklearn.metrics import accuracy_score, classification_report

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.25
              precision    recall  f1-score   support

Food & Dining       0.25      1.00      0.40         1
      Housing       0.00      0.00      0.00         1
     Shopping       0.00      0.00      0.00         1
Transportation      0.00      0.00      0.00         1

     accuracy                           0.25         4
    macro avg       0.06      0.25      0.10         4
 weighted avg       0.06      0.25      0.10         4

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predi
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

6.4 Saving the Model

Code Placement: After predictions.

```python
import pickle

with open("expense_model.pkl", "wb") as f:
    pickle.dump(model, f)
```

```python
files.download("expense_model.pkl")
```

7. Conclusion

The “Daily Life Optimizer: Personal Finance Classifier” successfully demonstrates how artificial intelligence and machine learning can simplify everyday financial management. By leveraging a **Naive Bayes classifier** with TF-IDF features, the model effectively categorizes expenses into meaningful segments such as Food, Travel, Bills, Shopping, Health, and Entertainment. The model achieved a satisfactory level of accuracy on the test dataset, indicating that even with a relatively small and domain-specific dataset, meaningful patterns in expense descriptions can be captured.

This project highlights several key insights:

1. **Real-world applicability:** Automating expense categorization reduces manual effort and improves financial awareness, directly benefiting users in daily life.
2. **Data-driven learning:** The system demonstrates that machine learning can adapt to localized contexts, recognizing specific vendors and spending patterns unique to a region.
3. **Scalability:** The approach used is scalable to larger datasets, allowing for continuous improvement and refinement as more transactional data is collected.

Overall, the project successfully integrates fundamental AI and ML concepts, showing a practical application that bridges technical learning with real-world utility.

8. Future Development

While the current implementation lays a solid foundation, several enhancements can be incorporated to improve functionality, accuracy, and user experience:

1. **Expanded Dataset:** Increasing the dataset to include thousands of transactions with diverse vendors and user habits would improve the model's robustness and generalization.
2. **Advanced Algorithms:** Experimenting with algorithms such as **Logistic Regression, Random Forest, or Gradient Boosting** could potentially increase accuracy and better handle complex patterns.
3. **Contextual Understanding:** Integrating **Natural Language Processing models like BERT or DistilBERT** could help classify ambiguous or multi-word expense descriptions more accurately.
4. **User Feedback Integration:** Allowing the model to learn from corrections (e.g., user reclassifications) could create a dynamic, self-improving system.
5. **Visualization & Insights:** Adding dashboards to provide insights like monthly spending trends, category-wise breakdowns, and budget alerts would enhance usability.
6. **Mobile or Web Integration:** Although the current scope is limited to a model, future development could include a **mobile or web interface** to make the system more accessible and interactive.

9. Final Remarks

This project demonstrates the **practical power of AI in daily life**, specifically in personal finance management. By automating expense categorization, it not only saves time but also empowers users to understand their spending habits better. The methodology is clear, reproducible, and serves as a strong foundation for future enhancements.

The experience gained from this project reinforces the importance of **data collection, preprocessing, model selection, and evaluation**—key pillars of any AI and ML project. It also emphasizes originality in applying AI to localized real-world problems, which is a vital skill in the industry.

In conclusion, the "Daily Life Optimizer" is more than a proof-of-concept; it is a practical demonstration of how machine learning can integrate seamlessly into daily routines, paving the way for smarter personal financial management and providing a template for further innovation.

10. References

- Scikit-learn Documentation
- GeeksforGeeks – Machine Learning Tutorials
- Research papers on Text Classification
- Python Official Documentation